5.3.9 Lesson Review

Date: 3/28/2026, 9:38:35 PM

Time Spent: 04:14

Score: 100%

Passing Score: 80%

Question 1 Correct

A DevSecOps team uses AI to monitor application logs and resource usage. The system flags requests that differ significantly from typical traffic patterns and alerts engineers for further review.

Which AI capability BEST describes this behavior?

- Static analysis
- Compliance auditing
- Anomaly detection ✓ Correct
- Automated remediation

Explanation

Anomaly detection is the correct answer. Anomaly detection is the capability that identifies events or behaviors that deviate from learned norms, such as unexpected request patterns or abnormal resource consumption. The AI system here is detecting these deviations and generating alerts, which is characteristic of anomaly detection.

Automated remediation is incorrect. Automated remediation refers to systems that take corrective actions, such as blocking an IP or rolling back a deployment. In this scenario, the primary function described is noticing unusual patterns and alerting engineers, not automatically fixing the issue.

Static analysis is incorrect. Static analysis examines code or configuration without executing it, looking for bugs or vulnerabilities. The example involves monitoring runtime logs and live usage patterns, which is outside the scope of static analysis.

Compliance auditing is incorrect. Compliance auditing checks whether processes and configurations adhere to regulatory or internal standards. While it may use automated tools, it is not focused on detecting unusual runtime behaviors in logs or traffic patterns as described in this scenario.

Related Content

5.3.7 AI in DevSecOps

resources\questions\q_ai_in_devsecops_10.question.xml

Question 2 ✓ Correct

How can a low-code platform MOST effectively support a security analyst who needs a customized log analysis report?

- It automatically blocks all suspicious traffic without any configuration options, reporting, or ability to customize how logs are interpreted for different environments and threat models.It supplies prebuilt log analysis workflows and lets the analyst tweak parameters, but does not permit adding or editing any underlying code for custom parsing or enrichment.
- It includes a visual query builder that generates code automatically and supports limited script injection, but requires most complex transformations to be handled by manually written standalone tools.
- It requires the analyst to design every aspect of the report by writing complex SQL queries and Python code from the ground up.It offers a dashboard designer with drag-and-drop widgets, while any advanced filtering or correlation logic must be implemented through full-length scripts managed outside the platform.
- (selected) It provides a visual report builder with templates and allows the analyst to add a small script to adjust how specific log fields are filtered and displayed. ✓ Correct

Explanation

Providing a visual report builder with templates and allowing the analyst to add a small script to adjust how specific log fields are filtered and displayed is the correct answer. This aligns with low-code principles: the bulk of the reporting workflow is created through visual tools, while small, targeted code snippets give the analyst flexibility to customize filters, formatting, or field handling.

Offering a dashboard designer with drag-and-drop widgets is incorrect. This reduces the role of the low-code environment to display only. Because all meaningful customization is pushed to external, fully coded scripts, the platform is not truly low-code for the core log-analysis logic.

Supplying prebuilt log analysis workflows and lets the analyst tweak parameters is incorrect. This describes a configuration-only or no-code style system. While parameters can be adjusted, the inability to inject custom code means it lacks the extensibility expected in a low-code environment.

Including a visual query builder that generates code automatically and supports limited script injection is incorrect. Although it appears similar, this design offloads the majority of complex logic to separate, fully coded tools, so the low-code aspects apply only to simple cases, not the main customization needs of log analysis.

Related Content

5.3.1 AI for Security Scripting and Content Summarization

resources\questions\q_ai_for_security_scripting_and_content_summarization_02.question.xml

Question 3 ✓ Correct

How do AI agents most effectively support continuous security in a CI/CD pipeline?

- By primarily optimizing pipeline performance metrics while providing basic alerts for irregular build or deployment behaviors.
- By integrating with testing and monitoring tools to identify vulnerabilities, enforce policies, and highlight risks during each stage of code delivery. ✓ Correct
- By focusing on validating infrastructure configurations in production environments rather than monitoring security throughout the entire pipeline.
- By automating routine security checks so that developers can prioritize rapid releases over detailed manual reviews.

Explanation

Integrating with testing and monitoring tools to identify vulnerabilities, enforce policies, and highlight risks during each stage of code delivery is the correct answer. AI agents best support continuous security when they are integrated into testing and monitoring across all pipeline stages. They detect vulnerabilities, enforce policies, and surface risks as code moves from development to production, aligning security with the pace of CI/CD.

Automating routine security checks is incorrect. While AI agents automate routine checks, their purpose is to maintain or improve security quality, not to trade it away for speed. Critical changes and ambiguous findings still require human review, so this answer overstates the shift away from manual oversight.

Primarily optimizing pipeline performance metrics while providing basic alerts is incorrect. Performance optimization and basic anomaly alerts are useful, but they do not capture the central security role of AI agents. In continuous security, agents must go beyond performance metrics to perform deeper analysis of code, dependencies, and configurations.

Focusing on validating infrastructure configurations in production environments is incorrect. Limiting AI agents to production configuration checks ignores the continuous aspect of security. They are most effective when used throughout the pipeline, detecting and preventing issues early rather than focusing mainly on the final production stage.

Related Content

5.3.7 AI in DevSecOps

resources\questions\q_ai_in_devsecops_07.question.xml

Question 4 ✓ Correct

In an AI-enhanced cloud environment, why might certain automated scaling or reconfiguration actions require human approval instead of being fully automated?

- Because changes that alter core network segments or security zones may need a human to validate potential side effects before deployment.
- Because any configuration update that improves performance should be postponed until a separate AI model verifies the expected cost savings.
- Because some high-impact changes must be manually authorized when usage exceeds preset limits, ensuring control over risk and cost. ✓ Correct
- Because adjustments to autoscaling policies should always be delayed until the AI can observe several weeks of additional traffic patterns.

Explanation

Because some high-impact changes must be manually authorized when usage exceeds preset limits, ensuring control over risk and cost is the correct answer. Once usage goes beyond predefined thresholds, human approval helps decide whether additional changes are justified given security, reliability, and budget.

Because changes that alter core network segments or security zones may need a human is incorrect. While such changes often do need review, the question emphasizes exceeding preset limits for scaling and reconfiguration, where overall risk and cost governance is the primary reason for human approval.

Because adjustments to autoscaling policies should always be delayed until the AI can observe several weeks of additional traffic patterns is incorrect. Historical data helps, but enforcing a fixed waiting period is not the main driver. The key issue is controlling significant changes that may have large operational or financial impact.

Because any configuration update that improves performance should be postponed is incorrect. Cost evaluation is useful, but human approval is mainly about overseeing high-impact changes beyond limits, not about requiring a secondary AI check before every performance improvement.

Related Content

5.3.4 AI in Security Workflows

resources\questions\q_ai_in_security_workflows_03.question.xml

Question 5 Correct

In an AI-assisted security workflow, which process helps analysts quickly extract key findings from large volumes of logs, alerts, and threat intelligence reports?

- Data enrichment
- Threat emulation
- Data summarization ✓ Correct
- Model deployment

Explanation

Data summarization is the correct answer. This is the process of condensing large, complex data sets into brief overviews, making it faster for analysts to see key patterns, issues, and priorities.

Threat emulation is incorrect. Threat emulation focuses on simulating attacker behavior to test defenses, not on condensing existing security data into shorter summaries.

Data enrichment is incorrect. Data enrichment adds context or additional fields to records, such as threat intel tags, but does not reduce or compress the data into concise findings.

Model deployment is incorrect. Model deployment is about putting AI models into production environments; it does not describe creating short, high-level summaries of security information.

Related Content

5.3.1 AI for Security Scripting and Content Summarization

resources\questions\q_ai_for_security_scripting_and_content_summarization_04.question.xml

Question 6 ✓ Correct

A financial services company is deploying an AI-driven security monitoring system to assist with incident detection and automated response. During pilot testing, the system starts flagging large volumes of legitimate backup traffic as potential exfiltration attempts and occasionally recommends blocking those connections.

The security architect decides this behavior must be addressed before enabling automated actions in production.

Which approach BEST applies implementation and training practices to improve the system's decision-making?

- (●) Retrain the model using labeled examples of normal and malicious traffic patterns, refine the definitions of legitimate backup activity, and adjust response rules so that high-impact actions require explicit human approval. ✓ Correct
- ( ) Introduce a supervised feedback loop where analysts review the AI's alerts on backup traffic, mark them as correct or incorrect, and rely on this feedback history to gradually influence the model's behavior without changing the current response logic.
- ( ) Add new rule-based exceptions for known backup IP ranges and schedules, allow the AI to continue scoring all other traffic as before, and plan to incorporate analyst feedback on misclassifications into a future tuning cycle.
- ( ) Retrain the model with additional traffic samples, but primarily lower the confidence threshold for classifying events as benign so that fewer false positives are raised, while keeping the existing automated response rules unchanged.

Explanation

Retraining the model using labeled examples of normal and malicious traffic patterns, refining the definitions of legitimate backup activity, and adjusting response rules so that high-impact actions require explicit human approval is the correct answer. This answer combines focused retraining with clear examples of normal vs. malicious patterns, explicitly models backup traffic, and tightens response rules so disruptive actions are gated by human approval. It directly improves both the model and how its outputs are used.

Retraining the model with additional traffic samples, but primarily lowering the confidence threshold for classifying events as benign, is incorrect. Although this option adds data, it mainly changes thresholds instead of teaching the model what backup traffic looks like. Keeping the same automated responses means the system can still make poor blocking recommendations, just on a slightly reduced alert set.

Adding new rule-based exceptions for known backup IP ranges and schedules is incorrect. Rule-based exceptions may cut immediate false positives but do not help the model learn backup behavior. The AI can still misclassify similar traffic outside the exception list, and deferring true tuning to a later cycle leaves the core problem unresolved.

Introducing a supervised feedback loop where analysts review the AI's alerts on backup traffic is incorrect. Analyst feedback is valuable, but using it without updating response logic or retraining is incomplete. The AI will keep producing the same risky recommendations while "slowly" adapting, instead of pairing feedback with structured model updates and safer automation controls.

Related Content

5.3.4 AI in Security Workflows

resources\questions\q_ai_in_security_workflows_05.question.xml

Question 7 Correct

A financial services company uses a CI/CD pipeline to deploy updates to its customer portal. During a recent incident, an automated scaling script opened more inbound ports than allowed by policy, exposing internal services.

Leadership now wants to tighten control over infrastructure changes while still maintaining fast deployments.The team decides to incorporate AI into their change-approval process.

Which approach should they implement?

- (●) AI-assisted approvals ✓ Correct
- ( ) Manual approvals with AI-generated deployment scripts
- ( ) Post-deployment anomaly alerts without approval controls
- ( ) Fully autonomous AI-enforced changes

Explanation

AI-assisted approvals is the correct answer. This uses AI to analyze proposed changes and recommend approve, deny, or escalate, while humans make the final decision. It speeds and standardizes reviews without removing human oversight.

Fully autonomous AI-enforced changes is incorrect. Here the AI directly approves and applies changes without human review. This increases speed but creates a single point of failure and can push risky configurations into production unchecked.

Manual approvals with AI-generated deployment scripts is incorrect. AI is used only to generate scripts or templates, not to assess risk. Change decisions remain fully manual, so misconfigurations can still be missed under time pressure.

Post-deployment anomaly alerts without approval controls is incorrect. AI monitors for problems only after deployment. This may detect bad changes later but does nothing to prevent unsafe configurations from being approved and released in the first place.

Related Content

5.3.7 AI in DevSecOps

resources\questions\q_ai_in_devsecops_17.question.xml

Question 8 ✓ Correct

A healthcare startup is rolling out an AI-assisted triage system that classifies incoming support tickets as urgent, normal, or low priority.

As part of its DevSecOps workflow, the team wants to validate that the latest model version does not expose sensitive information in its responses and makes consistent decisions across releases.

Which action BEST applies model testing to this requirement?

- (●) Implement a model testing stage that replays a fixed set of labeled test tickets, including some with deliberately injected sensitive data, compares outputs to expected labels and redaction rules, and blocks deployment if deviations exceed defined thresholds. ✓ Correct
- ( ) Deploy an AI-based analytics dashboard in production that tracks ticket volumes, average resolution times, model confidence scores, and team workload distribution, then suggests process changes to improve efficiency.
- ( ) Add an AI-powered dependency scanner that reviews all third-party libraries used by the triage system, correlating them with known vulnerabilities and outdated versions, and producing a prioritized remediation report.
- ( ) Configure an AI-augmented unit testing framework that generates test cases for the ticket routing microservice, ensuring it correctly calls the classification API and writes status changes to the database.

Explanation

Implementing a model testing stage that replays a fixed set of labeled test tickets, including some with deliberately injected sensitive data, compares outputs to expected labels and redaction rules, and blocks deployment if deviations exceed defined thresholds is the correct answer. This directly performs model testing by sending known, labeled, and sensitive test inputs to the model, verifying outputs against expected classifications and redaction behavior, and enforcing thresholds that can fail the pipeline if the model behaves incorrectly.

Configuring an AI-augmented unit testing framework that generates test cases for the ticket routing microservice is incorrect. This focuses on unit testing the surrounding microservice logic rather than validating the model's decisions or its handling of sensitive content, so it does not address the core need for model-focused testing.

Adding an AI-powered dependency scanner that reviews all third-party libraries used by the triage system is incorrect. This concentrates on dependency security through composition analysis and does not exercise the model's behavior or compare its outputs to expected results, which is central to model testing.

Deploying an AI-based analytics dashboard in production that tracks ticket volumes, average resolution times, model confidence scores, and team workload distribution is incorrect. This monitors operational metrics and suggests workflow optimizations after deployment, but it does not systematically test the model's outputs under controlled conditions before release.

Related Content

5.3.7 AI in DevSecOps

resources\questions\q_ai_in_devsecops_14.question.xml

Question 9 Correct

In an AI-supported DevSecOps workflow, which testing approach focuses on validating the behavior of individual code components before they are combined with other parts of the application?

- Performance testing
- Acceptance testing
- User interface testing
- Unit testing ✓ Correct

Explanation

Unit testing is the correct answer. Unit testing focuses on small, self-contained pieces of code and verifies they behave as expected before integration with other components. AI can assist by generating test cases for these units, helping detect defects early and supporting reliable automated pipelines.

Acceptance testing is incorrect. Acceptance testing determines whether the overall system meets business or user requirements, often near the end of the delivery process. It works at a high level, not at the level of individual functions or methods.

Performance testing is incorrect. Performance testing measures how the system behaves under various loads, such as response time or throughput. It examines system characteristics under stress, not the correctness of single, isolated code units.

User interface testing is incorrect. User interface testing evaluates the look, feel, and interaction behavior of the application's front end. It deals with screens, workflows, and user actions instead of directly validating individual low-level code components.

Related Content

5.3.7 AI in DevSecOps

resources\questions\q_ai_in_devsecops_04.question.xml

Question 10 ✓ Correct

A security operations team receives several gigabytes of data each day from firewall logs, IDS alerts, and endpoint protection reports. Analysts are falling behind on investigations because they spend hours manually reviewing raw entries to identify patterns and map them to known attack techniques.

The team wants to use an AI system to automatically ingest this data, highlight likely campaigns, map observed behaviors to a threat framework, and generate a concise summary report for human review at the start of each shift.

How should they configure the AI system to BEST meet this need?

- ○ Configure the AI system to cluster related log entries and alerts into incidents but leave the generation of written summaries and threat framework mappings to manual analyst workflows.
- ◉ Train the AI model to perform document summarization over combined log and alert data, including mapping indicators to known tactics, techniques, and procedures and generating attribution recommendations. ✓ Correct
- ○ Set up the AI model to generate per-source technical summaries for each tool independently, while correlation across sources and mapping to threat frameworks is performed by a separate manual review process.
- ○ Use the AI system to normalize and enrich all incoming security data with threat intelligence feeds, then rely on analysts to create shift reports and narrative summaries based on the enriched dataset.

Explanation

Training the AI model to perform document summarization over combined log and alert data, including mapping indicators to known tactics, techniques, and procedures and generating attribution recommendations is the correct answer. This option has the AI ingest and correlate security data, then generate structured summaries that include mappings to tactics, techniques, and attribution, directly reducing manual review and report writing.

Configuring the AI system to cluster related log entries and alerts into incidents is incorrect. Clustering incidents without generating written summaries or framework mappings still leaves analysts to do the most time-consuming synthesis work themselves.

Using the AI system to normalize and enrich all incoming security data with threat intelligence feeds is incorrect. Enrichment and normalization improve data quality but do not create the concise, shift-ready summaries the team needs; analysts still manually assemble reports.

Setting up the AI model to generate per-source technical summaries for each tool independently is incorrect. Separate per-source summaries do not provide a unified, cross-source view of campaigns, forcing analysts to manually correlate outputs and perform the threat framework mapping.

Related Content

5.3.1 AI for Security Scripting and Content Summarization

resources\questions\q_ai_for_security_scripting_and_content_summarization_07.question.xml